

Challenge Name: They Trusted the Numbers to Keep Their Secrets, but the Numbers Had Other Plans

Summary

The service implemented a toy Learning With Errors style encryption system. It exposed public samples generated with a fixed secret vector and then encrypted the flag bit-by-bit using the same secret.

The parameters were too small for security:

```
n = 50
q = 10007
secret coefficients in [-3, 3]
noise in {-1, 0, 1}
public sample budget = 320
```

The secret was recovered from public samples using a q-ary lattice and Kannan embedding. After recovering the secret, encrypted flag bits were decrypted by checking whether each residue was close to 0 or close to $q/2$.

The recovered flag is intentionally excluded from this report.

Target

```
nc 222.255.138.122 10026
```

Distributed archive:

```
lattice-in-the-machine-dist.tar.gz
```

Important files:

```
README.md
server.py
Dockerfile
wrapper.sh
docker-entrypoint.sh
```

Source Review

The service parameters were defined in `server.py` :

```
N = 50
Q = 10007
MAX_SAMPLES = 320
SECRET_BOUND = 3
NOISE_VALUES = (-1, 0, 1)
```

The secret was generated once per process:

```
_secret = _rng.integers(-SECRET_BOUND, SECRET_BOUND + 1, size=N,
dtype=np.int64)
```

A public sample was generated as:

```
def sample_pair():
    a = _rng.integers(0, Q, size=N, dtype=np.int64)
    e = int(_rng.choice(NOISE_VALUES))
    b = int((int(np.dot(a, _secret)) + e) % Q)
    return a, b
```

So every public sample satisfies:

$$b = \langle a, s \rangle + e \bmod q$$

For encrypted flag bits:

```
def encrypt_bit(bit):
    a, b = sample_pair()
    if bit:
        b = (b + Q // 2) % Q
    return a, b
```

Flag bytes were encoded least-significant-bit first:

```
for byte in flag.encode():
    for i in range(8):
        bits.append((byte >> i) & 1)
```

Solve Flow

1. Unpacked the distributed archive.
2. Read `server.py` and identified the LWE relation.
3. Confirmed the service exposes `params`, `sample`, and `encrypt`.
4. Collected 100 public samples, below the 320-sample limit.
5. Built a q-ary lattice from the sample matrix.
6. Converted the lattice to systematic form using 50 invertible sample rows.
7. Applied Kannan embedding by appending the target vector $(b, 1)$.
8. Ran LLL to recover the short error vector.
9. Solved $A*s = b - e \bmod q$ for the secret.
10. Verified every sample residual was in $\{-1, 0, 1\}$.
11. Requested the encrypted flag transcript.
12. Decrypted each bit by comparing $b - \langle a, s \rangle$ to 0 and $q/2$.
13. Reassembled bytes LSB-first.

Vulnerability

The construction leaks many equations using the same small secret:

$$A*s + e = b \bmod q$$

Both unknown parts are tiny:

```
s_i in [-3, 3]
e_i in {-1, 0, 1}
```

This makes the public vector b very close to the q-ary lattice point $A*s \bmod q$. Recovering the small error vector is a closest-vector problem. For these small challenge parameters, LLL on a Kannan embedding is enough.

Once the error vector is known, the secret is no longer hidden:

$$s = A^{-1} * (b - e) \bmod q$$

After recovering s , the encryption layer becomes trivial.

Lattice Method

Choose 50 linearly independent sample rows:

```
A0 = 50 x 50 invertible block
A1 = remaining rows
```

Compute:

```
C = A1 * A0^-1 mod q
```

This gives a systematic q-ary lattice:

```
(u, C*u + q*z)
```

The target vector is the collected b . Since:

```
b = A*s + e mod q
```

the vector b is close to a lattice vector by the small offset e .

Kannan embedding appends:

```
(b, 1)
```

After LLL, a row with final coordinate $+1$ or -1 and all small first coordinates reveals the error vector.

Decryption Logic

For each encrypted bit sample:

```
r = b - <a, s> mod q
```

Classification:

```
r in {0, 1, q-1}          -> bit 0
r in {q//2-1, q//2, q//2+1} -> bit 1
```

The ± 1 tolerance exactly matches the challenge noise.

Final Solve Script

```

#!/usr/bin/env python3
import random
import re
import socket

import numpy as np
from fpylll import IntegerMatrix, LLL

HOST = "222.255.138.122"
PORT = 10026
N = 50
Q = 10007
PAIR_RE = re.compile(r"A=([0-9,]+); b=([0-9]+)")

def centered(x):
    x = int(x) % Q
    return x - Q if x > Q // 2 else x

def parse_pairs(text):
    out = []
    for m in PAIR_RE.finditer(text):
        out.append(([int(x) for x in m.group(1).split(",")], int(m.group(2))))
    return out

def recv_until(sock, marker=b"> "):
    buf = b""
    while marker not in buf:
        chunk = sock.recv(4096)
        if not chunk:
            raise EOFError("closed")
        buf += chunk
    return buf.decode(errors="replace")

def cmd(sock, text):
    sock.sendall(text.encode() + b"\n")
    return recv_until(sock)

def inv_mod_mat(mat, q):
    mat = [[int(x) % q for x in row] for row in mat]
    n = len(mat)
    aug = [mat[i] + [1 if i == j else 0 for j in range(n)] for i in range(n)]

```

```

row = 0
for col in range(n):
    pivot = next((i for i in range(row, n) if aug[i][col] % q), None)
    if pivot is None:
        raise ValueError("singular")
    aug[row], aug[pivot] = aug[pivot], aug[row]
    inv = pow(aug[row][col], -1, q)
    aug[row] = [(x * inv) % q for x in aug[row]]
    for i in range(n):
        if i != row and aug[i][col] % q:
            f = aug[i][col] % q
            aug[i] = [(aug[i][j] - f * aug[row][j]) % q for j in range(2 *
n)]

    row += 1
return np.array([r[n:] for r in aug], dtype=object)

def recover_secret(samples):
    A = np.array([a for a, _ in samples], dtype=object)
    b = np.array([bb for _, bb in samples], dtype=object)
    m = len(samples)

    for attempt in range(1000):
        perm = list(range(m))
        random.Random(attempt).shuffle(perm)
        chosen = perm[:N]
        try:
            inv = inv_mod_mat(A[chosen, :], Q)
            break
        except ValueError:
            continue

    rest = [i for i in range(m) if i not in chosen]
    order = chosen + rest
    C = (A[rest, :].dot(inv)) % Q
    target = [int(b[i]) for i in order]

    basis = []
    for i in range(N):
        row = [0] * (m + 1)
        row[i] = 1
        for j in range(m - N):
            val = int(C[j, i])
            row[N + j] = val - Q if val > Q // 2 else val
        basis.append(row)
    for j in range(m - N):

```

```

        row = [0] * (m + 1)
        row[N + j] = Q
        basis.append(row)
basis.append(target + [1])

B = IntegerMatrix.from_matrix(basis)
LLL.reduction(B, delta=0.99)

err_ordered = None
for ri in range(B.nrows):
    row = [int(B[ri, c]) for c in range(B.ncols)]
    if abs(row[-1]) == 1:
        candidate = np.array(row[:-1], dtype=object)
        if row[-1] < 0:
            candidate = -candidate
        if all(int(x) in (-1, 0, 1) for x in candidate):
            err_ordered = candidate
            break

errors = np.zeros(m, dtype=object)
for pos, original in enumerate(order):
    errors[original] = err_ordered[pos]

rhs = (b[chosen] - errors[chosen]) % Q
secret = (inv.dot(rhs)) % Q
return np.array([centered(x) for x in secret], dtype=int)

def decrypt(secret, pairs):
    bits = []
    for a, b in pairs:
        r = (b - int(np.dot(np.array(a, dtype=object),
secret.astype(object)))) % Q
        if r in (0, 1, Q - 1):
            bits.append(0)
        elif r in (Q // 2 - 1, Q // 2, Q // 2 + 1):
            bits.append(1)
        else:
            raise RuntimeError(f"ambiguous residue {r}")

out = bytearray()
for i in range(0, len(bits), 8):
    value = 0
    for j, bit in enumerate(bits[i:i + 8]):
        value |= bit << j
    out.append(value)

```

```
    return out.decode(errors="replace")

sock = socket.create_connection((HOST, PORT), timeout=15)
recv_until(sock)
print(cmd(sock, "params"))

samples = []
for _ in range(100):
    samples.extend(parse_pairs(cmd(sock, "sample")))

secret = recover_secret(samples)
residuals = [centered(b - int(np.dot(np.array(a, dtype=object),
secret.astype(object)))) for a, b in samples]
assert all(x in (-1, 0, 1) for x in residuals)

encrypted = parse_pairs(cmd(sock, "encrypt"))
print(decrypt(secret, encrypted))
sock.sendall(b"quit\n")
```